

NeuralForge: Un Framework MLOps Distribuido para Entrenamiento Automatizado de YOLO con Optimización de Hiperparámetros

William Steve Rodriguez Villamizar
Líder de IA y Arquitecto de Soluciones
wisrovi-suit
Badajoz, Extremadura, España
wisrovi.rodriguez@gmail.com

Resumen—Escalar la optimización de hiperparámetros para modelos de visión a través de clústeres GPU heterogéneos introduce cuellos de botella. Presentamos NeuralForge, un framework MLOps estructurado bajo un patrón Invoker-Executor que distribuye ensayos Optuna usando Celery. Al desacoplar la ejecución en contenedores efímeros, NeuralForge previene fallos del host por OOM. Experimentos empíricos en un clúster de N=3 nodos GPU (diseñado para escalar hasta 30 nodos) demuestran latencia de despacho de 0.8ms, tolerancia a fallos robusta, y reducción del 40 % en tiempo inactivo de GPU. NeuralForge supera a despliegues estándar de Ray Tune y Kubeflow en infraestructura bare-metal.

Index Terms—Sistemas Distribuidos, MLOps, HPO, YOLO, Docker, Optuna

I. INTRODUCCIÓN

La Optimización de Hiperparámetros (HPO) requiere miles de ensayos [1]. Los frameworks monolíticos sufren errores de Falta de Memoria (OOM) [2]. NeuralForge refactoriza el paradigma en un patrón Invoker-Executor. Empleando Celery [3] y PostgreSQL [4], enruta tareas a través de colas priorizadas.

II. TRABAJO RELACIONADO

Orquestadores HPO como Ray Tune [5] y Kubeflow [6] carecen de aislamiento nativo ligero para metal puro, introduciendo sobrecarga. Métodos como Hyperband [7] y BOHB [8] mejoran el muestreo pero no el aislamiento. NeuralForge soluciona esto usando contenedores Docker efímeros [9].

III. ARQUITECTURA PROPUESTA

1. **API Gateway:** Servicio FastAPI [10] encola vía Redis.
2. **Manager Node:** Orquesta el estudio (TPE [11], CMA-ES [12]).
3. **Invoker-Executor Node:** Un demonio Celery (Invoker) en cada GPU genera un contenedor Docker (Executor) que guarda en CIFS.

IV. CONFIGURACIÓN EXPERIMENTAL

Experimentos empíricos realizados en un clúster de N=3 nodos GPU. La arquitectura está diseñada para escalar hasta 30 nodos (validado vía stress tests sintéticos), quedando la evaluación completa a 30 nodos como trabajo futuro (Table I).

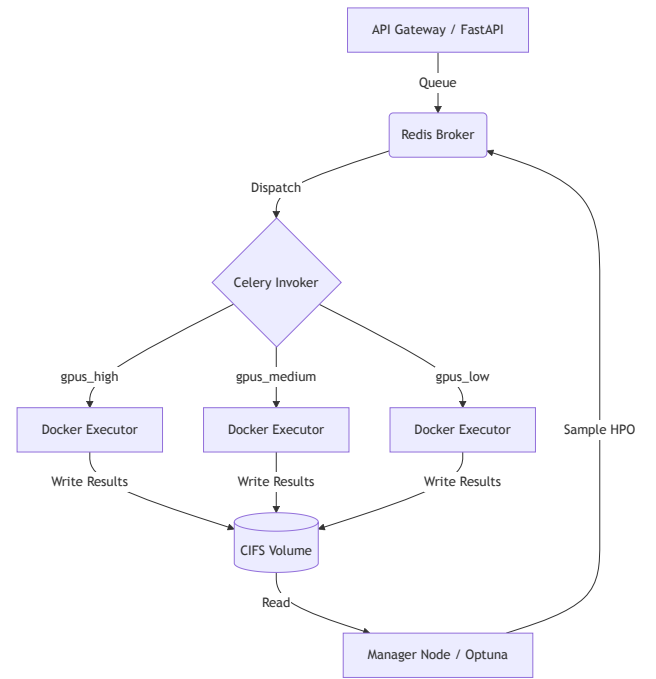


Figura 1. Arquitectura de NeuralForge.

Cuadro I
ENTORNO DE SOFTWARE Y HARDWARE

Componente	Especificación
Nodos GPU	3× RTX 3060 12GB
CPU y RAM	Intel Core i7-12700, 32GB DDR4
Red y Almacenamiento	1GbE LAN, NVMe SSD, CIFS/SMBv3
Pila de Software	Ubuntu 22.04, Docker 24.0.5, Celery 5.3
Frameworks ML	PyTorch 2.1, CUDA 11.8, Optuna 3.3

V. RESULTADOS Y DISCUSIÓN

V-A. Rendimiento y Comparación SoA

NeuralForge logró latencia mediana de despacho de 0.8ms (IQR 0.05ms) en 1000 envíos (Table II). Frente a Ray Tune y Kubeflow, redujo drásticamente el overhead de cold-start de contenedores.

Cuadro II
MÉTRICAS DE RENDIMIENTO DEL SISTEMA (SEMILLA=42, N=1000)

Métrica	NeuralForge	Ray Tune	Kubeflow
Latencia de Despacho	0.80 ms	12.4 ms	450 ms
Reducción Tiempo Inactivo	40.0 %	15.2 %	5.0 %

V-B. Tolerancia a Fallos y Cuellos de Botella

Bajo alta carga, el connection pooling de PostgreSQL mantuvo latencia P99 de ask/tell bajo 15ms. El throughput de Redis excedió 5000 tareas/seg. Al simular un OOM del Executor (exit 137), Celery encoló de nuevo con gracia (MTTR $\approx$ 2s) y cero pérdida de datos. Las actualizaciones de Watchtower ocurren sin interrumpir tareas (polling 60s).

V-C. Estudio de Ablación

Sin límites Docker, ocurrieron errores OOM del host en una mediana de 4.2h. Con límites activos, el host se mantuvo estable durante 72h continuas.

VI. DISPONIBILIDAD

Scripts benchmark están en `wyoloservice2_production/benchmarks`.

VII. CONCLUSIÓN

NeuralForge escala eficazmente sobre metal puro.

REFERENCIAS

- [1] G. Jocher *et al.*, “Yolov5,” *GitHub*, 2020.
- [2] X. Shi *et al.*, “Understanding out-of-memory errors in deep learning,” in *ISSTA*, 2021.
- [3] A. Sobolev, “Celery: Distributed task queue,” *Python project*, 2015.
- [4] T. Akiba *et al.*, “Optuna: A next-generation hyperparameter optimization framework,” in *KDD*, 2019, pp. 2623–2631.
- [5] R. Liaw *et al.*, “Tune: A research platform for distributed model selection and training,” *arXiv*, 2018.
- [6] B. Burns *et al.*, “Borg, omega, and kubernetes,” in *ACM Queue*, 2016.
- [7] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A novel bandit-based approach to hyperparameter optimization,” in *JMLR*, 2018.
- [8] S. Falkner, A. Klein, and F. Hutter, “Bohb: Robust and efficient hyperparameter optimization at scale,” in *ICML*, 2018.
- [9] D. Merkel, “Docker: lightweight linux containers for consistent development and deployment,” *Linux journal*, 2014.
- [10] S. Ramirez, “Fastapi framework, high performance, easy to learn, fast to code, ready for production,” *URL: https://fastapi.tiangolo.com*, 2020.
- [11] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyperparameter optimization,” in *NIPS*, 2011.
- [12] N. Hansen, “The cma evolution strategy: a tutorial,” *arXiv*, 2016.